

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY  
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY  
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**FINAL REPORT**

**10/05/2026**

**MACHINE LEARNING**

**GROUP 11**

**Supervisors:** Ph. D Truong Vinh Lan

——— o0o ———

**Students:** Tran Dang Khoa - 2252363  
Phan Tran Quoc Tuan - 2353273  
Tran Gia Lam - 2352670  
Nguyen Tan Hung - 2352439  
Nguyen Huu Cau - 2352129

HO CHI MINH CITY, MAY 10 2026

# Contents

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                      | <b>3</b>  |
| 1.1      | Group Work Assignment . . . . .                          | 3         |
| 1.2      | Problem Description . . . . .                            | 3         |
| 1.3      | Dataset . . . . .                                        | 3         |
| 1.4      | Objectives . . . . .                                     | 3         |
| 1.5      | Evaluation Metrics . . . . .                             | 3         |
| 1.5.1    | Regression Metrics . . . . .                             | 3         |
| 1.5.2    | Classification Metrics . . . . .                         | 4         |
| <b>2</b> | <b>Dataset Description</b>                               | <b>4</b>  |
| 2.1      | Source and Structure . . . . .                           | 4         |
| 2.2      | Feature Categories . . . . .                             | 4         |
| 2.3      | Target Variable: Log-Error . . . . .                     | 4         |
| <b>3</b> | <b>Exploratory Data Analysis</b>                         | <b>6</b>  |
| 3.1      | Target Distribution . . . . .                            | 6         |
| 3.2      | Missing Values . . . . .                                 | 6         |
| 3.3      | Feature Distributions . . . . .                          | 6         |
| 3.4      | Correlation Analysis . . . . .                           | 6         |
| 3.5      | Outlier Analysis . . . . .                               | 6         |
| <b>4</b> | <b>Data Preprocessing</b>                                | <b>7</b>  |
| 4.1      | Preprocessing Steps . . . . .                            | 7         |
| 4.2      | Final Feature Matrix . . . . .                           | 7         |
| <b>5</b> | <b>Machine Learning Pipeline</b>                         | <b>7</b>  |
| <b>6</b> | <b>Logistic Regression</b>                               | <b>9</b>  |
| 6.1      | Model Overview . . . . .                                 | 9         |
| 6.2      | Mathematical Formulation . . . . .                       | 9         |
| 6.2.1    | Binary Logistic Regression (Foundation) . . . . .        | 9         |
| 6.2.2    | Multinomial Logistic Regression . . . . .                | 9         |
| 6.2.3    | Loss Function and Regularisation . . . . .               | 9         |
| 6.3      | Feature Representation . . . . .                         | 10        |
| 6.4      | Class Imbalance Handling . . . . .                       | 10        |
| 6.5      | Regularisation Comparison . . . . .                      | 10        |
| 6.6      | Hyperparameter Optimisation . . . . .                    | 10        |
| 6.7      | Final Model Performance . . . . .                        | 10        |
| 6.8      | Per-class Performance and Coefficient Analysis . . . . . | 11        |
| <b>7</b> | <b>Random Forest</b>                                     | <b>12</b> |
| 7.1      | Model Overview . . . . .                                 | 12        |
| 7.2      | Justification . . . . .                                  | 12        |
| 7.3      | Hyperparameters . . . . .                                | 12        |
| 7.4      | Hyperparameter Optimisation with Optuna . . . . .        | 12        |
| 7.4.1    | Overview of Optuna . . . . .                             | 12        |
| 7.4.2    | Objective Function . . . . .                             | 13        |
| 7.4.3    | Search Space . . . . .                                   | 13        |
| 7.5      | Best Hyperparameters and Final Model . . . . .           | 13        |
| 7.6      | Model Performance . . . . .                              | 13        |
| <b>8</b> | <b>Support Vector Machine</b>                            | <b>14</b> |
| 8.1      | Model Overview . . . . .                                 | 14        |
| 8.2      | Mathematical Formulation . . . . .                       | 14        |
| 8.2.1    | Hard-Margin and Soft-Margin SVM . . . . .                | 14        |
| 8.2.2    | Kernel Trick and RBF Kernel . . . . .                    | 14        |
| 8.2.3    | Multiclass Strategy . . . . .                            | 14        |
| 8.3      | Model Variants . . . . .                                 | 14        |

|           |                                                       |           |
|-----------|-------------------------------------------------------|-----------|
| 8.4       | Hyperparameter Optimisation                           | 15        |
| 8.5       | Experimental Results                                  | 15        |
| 8.5.1     | Overall Performance                                   | 15        |
| 8.5.2     | Per-class Performance                                 | 15        |
| 8.5.3     | Support Vector Analysis                               | 15        |
| 8.6       | Feature Weight Analysis (LinearSVC)                   | 16        |
| <b>9</b>  | <b>Deep Learning</b>                                  | <b>17</b> |
| 9.1       | Model Overview                                        | 17        |
| 9.1.1     | Introduction to Deep Learning                         | 17        |
| 9.1.2     | Why Multi-Layer Perceptron (MLP)?                     | 17        |
| 9.2       | Mathematical Formulation                              | 17        |
| 9.2.1     | Feedforward Neural Network                            | 17        |
| 9.2.2     | Activation Function                                   | 18        |
| 9.2.3     | Loss Function                                         | 18        |
| 9.2.4     | Optimization Algorithm                                | 19        |
| 9.3       | Hyperparameter Tuning with Optuna                     | 19        |
| 9.3.1     | TPE Sampler                                           | 20        |
| 9.3.2     | Median Pruner                                         | 20        |
| 9.3.3     | Search Space                                          | 20        |
| 9.3.4     | Tuning Results                                        | 20        |
| 9.3.5     | Hyperparameter Sensitivity Analysis                   | 20        |
| 9.4       | Training Pipeline                                     | 21        |
| 9.4.1     | Data Split                                            | 21        |
| 9.4.2     | Training Loop                                         | 21        |
| 9.4.3     | Execution Environment                                 | 22        |
| 9.5       | Training Loss Analysis                                | 22        |
| 9.6       | Model Evaluation                                      | 23        |
| 9.6.1     | Test Set Results                                      | 23        |
| <b>10</b> | <b>Compare with Traditional ML</b>                    | <b>23</b> |
| 10.1      | Pipeline-Level Differences                            | 23        |
| 10.2      | Converting MLP Predictions to Classes                 | 24        |
| 10.3      | Quantitative Comparison                               | 24        |
| 10.4      | Analysis and Discussion                               | 25        |
| 10.4.1    | Overall difficulty of the task.                       | 25        |
| 10.4.2    | Traditional ML: accuracy vs. F1 macro trade-off.      | 25        |
| 10.4.3    | Deep learning: regression vs. classification framing. | 25        |
| <b>11</b> | <b>Conclusion</b>                                     | <b>26</b> |
| 11.1      | Task difficulty.                                      | 26        |
| 11.2      | Classification pipeline.                              | 26        |
| 11.3      | Regression pipeline.                                  | 26        |
| 11.4      | Deep learning vs. traditional Machine Learning.       | 26        |
| 11.5      | Limitations and Future Work                           | 26        |

# 1 Introduction

## 1.1 Group Work Assignment

| Student ID | Full Name           | Task                                        | Completion Rate |
|------------|---------------------|---------------------------------------------|-----------------|
| 2252363    | Tran Dang Khoa      | Random Forest model                         | 100%            |
| 2353273    | Phan Tran Quoc Tuan | Explolatory Data Analysis and preprocessing | 100%            |
| 2352670    | Tran Gia Lam        | Support Vector Machine Model                | 100%            |
| 2352439    | Nguyen Tan Hung     | Logistic Regression Model                   | 100%            |
| 2352129    | Nguyen Huu Cau      | Deep Learning                               | 100%            |

## 1.2 Problem Description

Automated Valuation Models (AVMs) play an important role in modern real-estate markets by providing estimation of property values without requiring a physical research. Zillow, one of the online real-estate platforms in the United States, operates a widely used AVM known as the *Zestimate*. The *Zestimate* is known to produce systematic errors in certain market segments, particularly for high-value properties and in regions with sparse comparable sales data. The Zillow Prize Kaggle competition challenges participants to improve upon the *Zestimate* by building a secondary model that predicts its residual error. This error, referred to as the *log-error*, is formally defined as:

$$\log_{\text{error}} = \log_{\text{Zestimate}} - \log_{\text{SalePrice}} \quad (1)$$

A positive log-error indicates that the *Zestimate* **overestimates** the true sale price, while a negative log-error indicates **underestimation**. The magnitude of the log-error reflects the degree of inaccuracy of the original model.

## 1.3 Dataset

The dataset originates from the Zillow Prize competition and consists of residential property transactions from Los Angeles County, California, spanning calendar years 2016 and 2017. It is formed by merging four source files including `train_2016_v2.csv`, `train_2017.csv`, `properties_2016.csv`, and `properties_2017.csv` on the unique property identifier `parcelid`. The resulting merged dataset contains **167,888 transaction records** and **61 feature columns** covering structural property attributes (e.g: bedroom count, square footage, year built), geographic identifiers (e.g: latitude, longitude, FIPS code, census tract), tax assessments, and transaction metadata.

## 1.4 Objectives

The objective of this project is to build supervised machine learning models that can predict the Zillow log-error for residential properties based on a set of property attributes. The project pursues two parallel formulations:

1. **Regression formulation:** Predict the continuous log-error value directly, evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination  $R^2$ .
2. **Classification formulation:** Discretise the log-error into three ordinal classes: *underestimate* (Class 0), *accurate* (Class 1), and *overestimate* (Class 2), and train multi-class classifiers to predict the category of each transaction. Performance is evaluated using Accuracy and Macro-averaged F1-score.

## 1.5 Evaluation Metrics

### 1.5.1 Regression Metrics

- **Mean Absolute Error (MAE):**  $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$ . MAE is the primary regression metric as it is compatible to outliers and directly interpretable in the units of the log-error.
- **Root Mean Squared Error (RMSE):**  $RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$ . RMSE penalises large errors more heavily than MAE, making it sensitive to outlier predictions.
- **Coefficient of Determination ( $R^2$ ):** Measures the proportion of variance in the target explained by the model. A value of  $R^2 > 0$  indicates that the model is more informative than the trivial mean predictor.

### 1.5.2 Classification Metrics

- **Accuracy:** The fraction of correctly classified samples. In this study, accuracy is a supplementary metric because the *accurate* class accounts for approximately 50% of the data, making accuracy susceptible to majority-class bias.
- **Macro-averaged F1-score (Macro-F1):**  $F1_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K F1_k$ , where  $K = 3$  is the number of classes. Macro-F1 assigns equal weight to each class regardless of support, making it the **primary classification metric** in this study. It penalises models that ignore the minority underestimate and overestimate classes.
- **Per-class Precision, Recall, and F1:** Reported to provide an overview of per-class model behaviour.

## 2 Dataset Description

### 2.1 Source and Structure

The dataset is derived from the publicly available Zillow Prize competition on Kaggle. It is constructed by merging property attributes from two annual property files with transaction-level log-error files. The key attributes of the merged dataset are summarised in Table ??.

| Property                     | Value                          |
|------------------------------|--------------------------------|
| Total records                | 167,888                        |
| Original columns             | 61                             |
| Numeric columns              | 53                             |
| Categorical (object) columns | 6                              |
| Transaction years            | 2016, 2017                     |
| Geographic scope             | Los Angeles County, California |
| Target variable              | logerror (continuous)          |
| Target mean                  | $\approx 0.014$                |
| Target standard deviation    | $\approx 0.166$                |
| Target range                 | $[-4.66, +5.26]$               |

**Table 2:** Summary of the Zillow merged dataset.

### 2.2 Feature Categories

The 61 original columns span the following semantic categories:

- **Structural attributes:** Bedroom count (bedroomcnt), bathroom count (bathroomcnt), calculated finished square footage (calculatedfinishedsquarefeet), lot size (lotsizesquarefeet), year built (yearbuilt).
- **Tax and valuation:** Total tax value (taxvaluedollarcnt), structure tax value (structuretaxvaluedollarcnt), land tax value (landtaxvaluedollarcnt), tax amount (taxamount).
- **Geographic identifiers:** Latitude, longitude, FIPS code, census tract and block, raw census tract, property county land use code, property zoning description.
- **Transaction metadata:** Transaction date (transactiondate), transaction year, month, and quarter (engineered).
- **Property type indicators:** Hot tub/spa flag, tax delinquency flag, heating system type, air conditioning type, deck type.

### 2.3 Target Variable: Log-Error

The target variable logerror is a continuous real-valued quantity. Its distribution is approximately Gaussian and centre near zero (mean  $\approx 0.014$ ), reflecting that the Zestimate is well-calibrated on average. However, the distribution exhibits heavy tails, with values extending to  $\pm 5$ , indicating occasional severe valuation errors. These extreme observations correspond to unusual properties or market conditions that the Zestimate is particularly ill-suited to model.

For the classification formulation, the continuous target is discretised into three ordinal classes using the first and third quartile values computed from the training set:

$$y = \begin{cases} 0 & \text{(underestimate), if } \logerror < Q_1 = -0.02526 \\ 1 & \text{(accurate), if } Q_1 \leq \logerror \leq Q_3 = 0.03920 \\ 2 & \text{(overestimate), if } \logerror > Q_3 \end{cases} \quad (2)$$

This yields a moderately imbalanced three-class distribution: approximately 25% underestimate, 50% accurate, and 25% overestimate.

### 3 Exploratory Data Analysis

A systematic Exploratory Data Analysis (EDA) was conducted prior to model development to understand the dataset structure, identify quality issues, and guide preprocessing decisions. The analysis was carried out in a shared Jupyter notebook (`src/EDA+preprocessing.ipynb`) on the merged Zillow dataset.

#### 3.1 Target Distribution

The log-error distribution is approximately zero-centred and bell-shaped, consistent with the Gaussian assumption. The mean log-error of  $\approx 0.014$  indicates a slight tendency for the Zestimate to overestimate property values on average. The presence of extreme outliers on both tails (beyond  $\pm 1.0$ ) suggests that a small fraction of properties are particularly difficult for the Zestimate to value accurately.

#### 3.2 Missing Values

A missing value analysis was performed for every column, computing the missing count, missing percentage, data type, and number of unique values. Thirteen columns were found to exceed a 98% missing rate and were subsequently removed from the dataset. Examples of these highly sparse columns include:

- `basementsqft` – basement square footage
- `buildingclasstypeid` – building class type identifier
- `storytypeid` – story type identifier
- `fireplaceflag` – fireplace indicator

The top 20 columns by missing rate were visualised using a horizontal bar chart to facilitate prioritised removal decisions.

#### 3.3 Feature Distributions

Histograms were plotted for key numeric features including **`bathroomcnt`**, **`bedroomcnt`**, **`calculatedfinishedsquarefeet`**, **`lotssquarefeet`**, **`yearbuilt`**, **`landtaxvaluedollarcnt`**, **`structuretaxvaluedollarcnt`**, **`taxvaluedollarcnt`**, and **`taxamount`**. The analysis revealed that many features exhibit right skewness and large dynamic ranges, underscoring the importance of standard scaling prior to model training.

#### 3.4 Correlation Analysis

Pearson correlations between all numeric features and the target `logerror` were computed. A correlation heatmap was produced for a predefined set of key candidate features alongside the target. The key finding is that individual feature correlations with the log-error are generally **very low** (absolute values below 0.05), suggesting that:

1. The log-error signal is diffuse and not dominated by any single property attribute.
2. Linear models are likely to underfit without extensive feature engineering.
3. Ensemble methods capable of capturing non-linear feature interactions are well-suited to the problem.

#### 3.5 Outlier Analysis

The Interquartile Range (IQR) method was applied to count outliers in each numeric column and report the outlier percentage among non-null values. Several columns exhibit large outlier proportions. Notable examples include:

- `garagecarcnt`: 26.8% outlier rate
- `roomcnt`: 22.8% outlier rate
- `bathroomcnt`: significant proportion of extreme values

These findings informed the decision to apply Winsorisation during preprocessing.

## 4 Data Preprocessing

All preprocessing steps were implemented using a `scikit-learn` pipeline composed of `ColumnTransformer` and `Pipeline` objects, ensuring that no data leakage occurs between training and testing sets. The pipeline was applied consistently across all models.

### 4.1 Preprocessing Steps

The following steps were applied sequentially:

1. **Duplicate removal:** No duplicate rows were found in the merged dataset.
2. **Temporal feature extraction:** The raw `transactiondate` column was parsed to extract `transaction_year`, `transaction_month`, and `transaction_quarter` as new numeric features.
3. **High-missingness column removal:** Thirteen columns with a missing rate exceeding 98% were dropped, as they are unlikely to contribute predictive information.
4. **Single-value column removal:** Columns with only one unique value were removed, as they carry no discriminative information.
5. **Outlier handling:** Winsorisation was applied to all numeric features, clipping values at the 1st and 99th percentiles to reduce the influence of extreme observations.
6. **Target and identifier separation:** The target variable `logerror` and the identifier column `parcelid` were isolated prior to feature engineering.
7. **Train/test split:** A random 80/20 split was applied with `random_state=42`, yielding 134,310 training samples and 33,578 test samples. The scaler and all encoding statistics were fitted exclusively on the training set.
8. **Frequency encoding:** High-cardinality categorical columns (`propertycountylandusecode`, `propertyzoningdesc`; more than 20 unique values) were replaced with their relative frequency computed from the training set.
9. **Numeric pipeline:** Median imputation followed by standard scaling (`StandardScaler`: zero mean, unit variance).
10. **Categorical pipeline:** Mode imputation followed by one-hot encoding (`OneHotEncoder` with `handle_unknown='ignore'`) for low-cardinality features (`hashottuborspa`, `taxdelinquencyflag`).

### 4.2 Final Feature Matrix

After all preprocessing steps, the final feature matrix contains **49 features**: 47 numeric features (including frequency-encoded high-cardinality categoricals) and 2 binary one-hot encoded features.

**Table 3:** *Dataset split summary after preprocessing.*

| Partition    | Samples | Proportion |
|--------------|---------|------------|
| Training set | 134,310 | 80%        |
| Test set     | 33,578  | 20%        |
| Total        | 167,888 | 100%       |

## 5 Machine Learning Pipeline

Three model families were selected for this study, each with a distinct modelling philosophy:

1. **Random Forest:** An ensemble regression model capable of capturing non-linear interactions in high-dimensional tabular data.
2. **Logistic Regression:** A parametric linear classifier providing a transparent, interpretable baseline for the three-class task.



3. **Support Vector Machine (SVM):** A margin-based classifier evaluated under both linear and non-linear (RBF kernel) configurations.
4. **Deep Learning:** Compare performance with traditional Machine Learning models to assess potential improvements.

## 6 Logistic Regression

### 6.1 Model Overview

Logistic Regression is a fundamental parametric supervised learning algorithm that, despite its name, is a classification model. It predicts class membership probabilities via a linear combination of features passed through a non-linear activation function. For the three-class classification task ( $K = 3$ : underestimate, accurate, overestimate), **Multinomial Logistic Regression** with a Softmax output is employed.

Key characteristics that motivate its use as a baseline include:

- **Interpretability:** Feature coefficients directly quantify the effect of each predictor on each class, enabling domain validation.
- **Efficiency:** Training is a convex optimisation problem with a guaranteed global optimum, and convergence typically occurs in under one minute.
- **Probabilistic outputs:** Class probability estimates are natively available and useful for downstream decision-making.
- **Robustness:** Regularisation mitigates the effect of multicollinearity.

### 6.2 Mathematical Formulation

#### 6.2.1 Binary Logistic Regression (Foundation)

For a binary problem with classes  $\{0, 1\}$ , the hypothesis function is:

$$h(\mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \mathbf{w}^T \mathbf{x} + b \quad (3)$$

where  $\sigma(\cdot)$  is the sigmoid function mapping  $z$  to the interval  $[0, 1]$ .

#### 6.2.2 Multinomial Logistic Regression

For  $K = 3$  classes, the Softmax function is used to produce calibrated class probabilities:

$$P(y = k | \mathbf{x}) = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}, \quad z_k = \mathbf{w}_k^T \mathbf{x} + b_k \quad (4)$$

The predicted class is selected via:

$$\hat{y} = \arg \max_k P(y = k | \mathbf{x}) \quad (5)$$

#### 6.2.3 Loss Function and Regularisation

The model minimises the categorical cross-entropy loss:

$$\mathcal{L}(\mathbf{w}) = - \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log(p_{ik}) \quad (6)$$

Two regularisation variants are compared. L1 (Lasso) regularisation promotes coefficient sparsity:

$$\mathcal{L}_{\text{reg}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \frac{1}{C} \sum_{k=1}^K \|\mathbf{w}_k\|_1 \quad (7)$$

L2 (Ridge) regularisation penalises large weight magnitudes:

$$\mathcal{L}_{\text{reg}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \frac{1}{2C} \sum_{k=1}^K \|\mathbf{w}_k\|^2 \quad (8)$$

where a larger  $C$  corresponds to weaker regularisation.

## 6.3 Feature Representation

The model operates on the full **50-feature** preprocessed matrix produced by the shared preprocessing pipeline (47 numeric features after standard scaling plus 3 additional encoded categorical features). `StandardScaler` ensures zero mean and unit variance for all numeric inputs, which is critical because unscaled high-magnitude features (e.g., `taxvaluedollarcnt`) would otherwise dominate gradient updates.

## 6.4 Class Imbalance Handling

The three-class target distribution is moderately imbalanced (approximately 50% accurate, 25% each for underestimates and overestimates). Without class balancing, the baseline model achieves a test macro-F1 of only 0.31, as it predominantly predicts the majority “accurate” class. Setting `class_weight=‘balanced’` adjusts the per-sample loss weight inversely proportional to class frequency:

$$w_k = \frac{n_{\text{samples}}}{n_{\text{classes}} \cdot n_{\text{samples},k}} \quad (9)$$

This adjustment, combined with Optuna hyperparameter optimisation, enabled the tuned model to reach a macro-F1 of 0.4108, a **+32.77% relative improvement** over the unweighted baseline.

## 6.5 Regularisation Comparison

Four regularisation strategies were evaluated on the test set with `C=1.0` and **without class weighting** to isolate the effect of regularisation type alone. Results are shown in Table 4.

**Table 4:** Effect of regularisation type on test performance (no class weighting,  $C = 1.0$ ).

| Regularisation                     | Test Accuracy | F1-macro      | ROC-AUC       |
|------------------------------------|---------------|---------------|---------------|
| L2 – Baseline (lbfgs, $C = 1.0$ )  | 50.69%        | 0.3094        | 0.6012        |
| L1 ( $C = 1.0$ )                   | 50.75%        | 0.3106        | 0.6015        |
| ElasticNet ( $C = 1.0$ )           | 50.75%        | 0.3106        | 0.6016        |
| No regularisation                  | 50.70%        | 0.3096        | 0.6012        |
| <b>Optuna-tuned (L1, balanced)</b> | <b>44.65%</b> | <b>0.4108</b> | <b>0.6007</b> |

All unweighted configurations yield nearly identical macro-F1 scores ( $\approx 0.31$ ) regardless of regularisation type, confirming that class balancing—not the choice of penalty—is the primary driver of performance improvement.

## 6.6 Hyperparameter Optimisation

Optuna with TPE sampling was used over **30 trials** and 5-fold cross-validation. The search space included  $C$  (log-uniform), solver (lbfgs, saga), penalty type (L1 / L2 / ElasticNet), class weight strategy, and an optional PCA dimensionality reduction step.

The best configuration found (**Trial #2**) was:

- Solver: saga
- Penalty: L1
- $C = 18.663$
- `class_weight`: balanced
- PCA: not applied

The best cross-validated macro-F1 was **0.4072**.

## 6.7 Final Model Performance

The final model was retrained on the full 134,310-sample training set and evaluated on the 33,578-sample held-out test set. Results are reported in Table 5.

**Table 5:** Final Logistic Regression test-set performance (Optuna-tuned, Trial #2).

| Metric             | Value   |
|--------------------|---------|
| Test Accuracy      | 44.65%  |
| Precision (macro)  | 0.4128  |
| Recall (macro)     | 0.4139  |
| F1 (macro)         | 0.4108  |
| F1 (weighted)      | 0.4493  |
| ROC-AUC (OvR)      | 0.6007  |
| Gain over baseline | +32.77% |

## 6.8 Per-class Performance and Coefficient Analysis

Per-class metrics on the full test set ( $n = 33,578$ ) are reported in Table 6. The underestimate class has the lowest precision (0.32), while the overestimate class has the lowest F1-score (0.31). Both minority classes are substantially harder to identify than the “accurate” majority class.

**Table 6:** Per-class metrics for the final Logistic Regression model (test set,  $n = 33,578$ ).

| Class             | Precision | Recall | F1   | Support |
|-------------------|-----------|--------|------|---------|
| 0 (Underestimate) | 0.32      | 0.41   | 0.36 | 8,440   |
| 1 (Accurate)      | 0.59      | 0.54   | 0.56 | 16,787  |
| 2 (Overestimate)  | 0.33      | 0.29   | 0.31 | 8,351   |
| Macro average     | 0.41      | 0.41   | 0.41 | 33,578  |

The top-5 features by mean absolute coefficient magnitude (averaged across all three class weight vectors  $\mathbf{w}_k$ ) are: (1) `taxvaluedollarcnt` (0.1756), (2) `yearbuilt` (0.1374), (3) `censustractandblock` (0.1359), (4) `taxamount` (0.1332), and (5) `hashottuborspa / taxdelinquencyflag_Y` (0.1048 each). Property tax value, building age, location identifiers, and tax burden dominate predictive importance, reflecting that Zillow’s valuation errors are driven by a combination of property valuation, temporal, and geographic factors.

## 7 Random Forest

### 7.1 Model Overview

A Random Forest is an ensemble learning method that aggregates the predictions of multiple decorrelated decision trees. It combines two variance-reduction mechanisms: *bagging* (Bootstrap Aggregating), in which each tree is trained on an independent bootstrap sample of the training data, and *random feature subspace selection*, in which only a random subset of features is considered at each split node.

For regression, the ensemble prediction is the arithmetic mean of all tree predictions:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T \hat{y}_t(\mathbf{x}) \quad (10)$$

where  $T$  is the number of trees and  $\hat{y}_t(\mathbf{x})$  is the prediction of the  $t$ -th tree for input  $\mathbf{x}$ .

### 7.2 Justification

The Random Forest is particularly well-suited to the Zillow log-error problem for the following reasons:

- **High dimensionality:** The 49-feature input space is handled naturally without explicit feature selection.
- **Non-linear relationships:** Correlations observed during EDA are weak and linear, suggesting that linear models would underfit. Tree-based ensembles can capture complex non-linear interactions.
- **Mixed feature types:** Decision tree splits are agnostic to feature scale, reducing sensitivity to imperfect standardisation.
- **Robustness to outliers:** The ensemble mechanism and split-based criteria reduce sensitivity to the residual outliers present in the dataset.
- **Built-in feature importance:** Impurity-based feature importance scores enable interpretability and potential feature engineering.

### 7.3 Hyperparameters

The key hyperparameters of the Random Forest Regressor and their roles are described in Table 7.

Table 7: Key hyperparameters of the Random Forest Regressor.

| Hyperparameter    | Type           | Role and Impact                                                                   |
|-------------------|----------------|-----------------------------------------------------------------------------------|
| n_estimators      | Integer        | Number of trees. More trees improve stability and reduce variance.                |
| max_depth         | Integer / None | Maximum tree depth. Limits model complexity.                                      |
| min_samples_split | Integer        | Minimum samples required to split an internal node.                               |
| min_samples_leaf  | Integer        | Minimum samples required in a leaf node.                                          |
| max_features      | String / Float | Features considered at each split (sqrt, log2, or fraction). Controls randomness. |
| bootstrap         | Boolean        | If True, uses bootstrap sampling; enables out-of-bag error estimation.            |
| random_state      | Integer        | Seed for reproducibility. Fixed at 42.                                            |
| n_jobs            | Integer        | CPU cores for parallel training. Set to -1.                                       |

The baseline model was initialised with `n_estimators=100` and all remaining parameters at their `scikit-learn` defaults.

### 7.4 Hyperparameter Optimisation with Optuna

#### 7.4.1 Overview of Optuna

Optuna is an automatic hyperparameter optimisation framework that implements Bayesian optimisation via the Tree-structured Parzen Estimator (TPE) sampler. Unlike grid or random search, TPE builds a probabilistic model of the objective function and focuses subsequent trials on promising regions of the hyperparameter space.

## 7.4.2 Objective Function

The objective function evaluates a candidate hyperparameter configuration  $\theta$  by training a Random Forest on the training set and returning the three-fold cross-validated MAE:

$$\mathcal{L}(\theta) = \frac{1}{K} \sum_{k=1}^K \text{MAE}(y_k, \hat{y}_k(\theta)) \quad (11)$$

where  $K = 3$ . The study is configured with `direction='minimize'`.

## 7.4.3 Search Space

The Optuna search space is defined in Table 8.

**Table 8:** Optuna hyperparameter search space for the Random Forest.

| Hyperparameter    | Search Type       | Range / Choices        |
|-------------------|-------------------|------------------------|
| n_estimators      | Integer (step 50) | [50, 300]              |
| max_depth         | Integer           | [3, 20]                |
| min_samples_split | Integer           | [2, 20]                |
| min_samples_leaf  | Integer           | [1, 10]                |
| max_features      | Categorical       | {sqrt, log2, 0.3, 0.5} |
| bootstrap         | Categorical       | {True, False}          |

The study was executed for 30 trials over the full training set of 134,310 samples with `n_jobs=-1` to minimise wall-clock time.

## 7.5 Best Hyperparameters and Final Model

After 30 trials, Optuna identified **Trial #22** as the optimal configuration, achieving a cross-validated MAE of 0.068851. The best hyperparameters are reported in Table 9.

**Table 9:** Best hyperparameters identified by Optuna (Trial #22).

| Hyperparameter    | Best Value |
|-------------------|------------|
| n_estimators      | 200        |
| max_depth         | 7          |
| min_samples_split | 11         |
| min_samples_leaf  | 8          |
| max_features      | log2       |
| bootstrap         | True       |
| random_state      | 42         |
| n_jobs            | -1         |

## 7.6 Model Performance

The final Random Forest was retrained on the full training set using the optimal hyperparameters and evaluated on the held-out test set. Results are reported in Table 10.

**Table 10:** Comparison of Baseline and Optuna-Tuned Random Forest.

| Model                                | MAE             | RMSE            | $R^2$           |
|--------------------------------------|-----------------|-----------------|-----------------|
| Baseline RF (default, $T = 100$ )    | 0.075422        | 0.170595        | -0.014465       |
| Optuna-tuned RF ( $T = 200$ , tuned) | <b>0.069765</b> | <b>0.168629</b> | <b>0.008790</b> |
| Improvement                          | +7.50%          | +1.15%          | —               |

The Optuna-tuned model achieves a **7.50% reduction in MAE** and a **1.15% reduction in RMSE** over the baseline. Crucially, the tuned model flips the  $R^2$  coefficient from negative ( $-0.014$ ) to positive ( $+0.009$ ), confirming that the tuned model is genuinely predictive relative to the trivial mean predictor, whereas the default model was not.

## 8 Support Vector Machine

### 8.1 Model Overview

A Support Vector Machine (SVM) is a discriminative classifier that searches for the decision boundary which maximises the margin between classes. Rather than minimising classification error alone, SVM also maximises the geometric distance between the hyperplane and the nearest training points—the *support vectors*. This margin-maximisation principle is theoretically linked to improved generalisation performance.

Key characteristics of SVM include:

- **Maximum-margin principle:** The decision boundary is positioned to maximise the separation between classes.
- **Kernel methods:** Non-linear decision boundaries can be induced via kernel functions (e.g., the Radial Basis Function kernel) without explicitly computing the high-dimensional feature transformation.
- **Regularisation:** The parameter  $C$  controls the bias-variance trade-off between margin width and training error tolerance.

### 8.2 Mathematical Formulation

#### 8.2.1 Hard-Margin and Soft-Margin SVM

For binary classification with labels  $y_i \in \{-1, +1\}$ , the hard-margin SVM solves:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad (12)$$

In practice, classes are not perfectly separable. The soft-margin SVM introduces slack variables  $\xi_i \geq 0$ :

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \quad (13)$$

#### 8.2.2 Kernel Trick and RBF Kernel

To capture non-linear patterns, the decision function is expressed in terms of a kernel function  $K(\mathbf{x}_i, \mathbf{x}_j)$ :

$$f(\mathbf{x}) = \sum_{i=1}^N \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b \quad (14)$$

The Radial Basis Function (RBF) kernel is defined as:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (15)$$

where  $\gamma$  controls the locality of the kernel. Large  $\gamma$  produces a more localised, flexible boundary; small  $\gamma$  yields a smoother boundary.

#### 8.2.3 Multiclass Strategy

LinearSVC employs the *one-vs-rest* (OvR) strategy, training one binary classifier per class. Kernel-based SVC uses the *one-vs-one* (OvO) strategy, training  $\binom{K}{2} = 3$  pairwise binary classifiers for  $K = 3$  classes.

### 8.3 Model Variants

Five SVM configurations were evaluated:

1. **Dummy Baseline:** Majority-class classifier. Always predicts the “accurate” class. Establishes the lower bound for evaluation.
2. **LinearSVC:** Trained on the full training set (134,310 samples). Provides a scalable linear baseline.
3. **Exact RBF SVC:** Trained on a stratified 20,000-sample subset due to the  $O(n^2)$ – $O(n^3)$  computational cost of exact kernel SVM. Evaluated on the full test set.

4. **Optuna Tuned Exact RBF SVC:** Exact RBF SVC retrained with hyperparameters selected by Optuna on the same 20,000-sample stratified subset.
5. **Nyström RBF Approximation + LinearSVC:** Uses the Nyström method to construct an explicit low-dimensional approximation of the RBF feature map,  $K(X, X) \approx \Phi(X)\Phi(X)^T$ , after which LinearSVC is applied. Trained on the full 134,310-sample training set.

## 8.4 Hyperparameter Optimisation

Optuna was used to jointly optimise the model family, regularisation parameter  $C$ , RBF kernel parameter  $\gamma$ , and class weighting strategy, maximising 3-fold stratified cross-validation macro-F1 on a 20,000-sample optimisation subset.

The best configuration found by Optuna was:

- **family:** Exact RBF SVC (subset)
- $C = 0.7073$
- $\gamma = 0.02237$
- **class\_weight:** balanced

The best cross-validation macro-F1 was 0.4116.

## 8.5 Experimental Results

### 8.5.1 Overall Performance

All SVM model results are reported in Table 11.

**Table 11:** Experimental results of SVM model variants.

| Model                      | Train Size | Time (s) | Train Acc. | Test Acc.     | Precision | Recall | F1-macro      |
|----------------------------|------------|----------|------------|---------------|-----------|--------|---------------|
| Nyström RBF + LinearSVC    | 134,310    | 181.64   | 0.5035     | <b>0.5017</b> | 0.4405    | 0.4224 | <b>0.4212</b> |
| Exact RBF SVC              | 20,000     | 101.97   | 0.5014     | 0.4499        | 0.4224    | 0.4240 | 0.4204        |
| Optuna Tuned Exact RBF SVC | 20,000     | 104.23   | 0.4919     | 0.4503        | 0.4226    | 0.4237 | 0.4200        |
| LinearSVC                  | 134,310    | 75.65    | 0.4979     | 0.4962        | 0.4247    | 0.4023 | 0.3936        |
| Dummy Baseline             | 134,310    | 0.01     | 0.5024     | 0.4999        | 0.1666    | 0.3333 | 0.2222        |

### 8.5.2 Per-class Performance

The exact RBF SVC improves recall for the two minority classes substantially over LinearSVC (Table 12), explaining its superior macro-F1 despite lower overall accuracy.

**Table 12:** Per-class metrics of the three main SVM variants.

| Model                   | Class         | Precision | Recall | F1   | Support |
|-------------------------|---------------|-----------|--------|------|---------|
| LinearSVC               | Underestimate | 0.35      | 0.25   | 0.29 | 8,440   |
|                         | Accurate      | 0.56      | 0.78   | 0.65 | 16,787  |
|                         | Overestimate  | 0.37      | 0.18   | 0.24 | 8,351   |
| Exact RBF SVC           | Underestimate | 0.33      | 0.42   | 0.37 | 8,440   |
|                         | Accurate      | 0.60      | 0.53   | 0.56 | 16,787  |
|                         | Overestimate  | 0.34      | 0.32   | 0.33 | 8,351   |
| Nyström RBF + LinearSVC | Underestimate | 0.38      | 0.27   | 0.32 | 8,440   |
|                         | Accurate      | 0.57      | 0.74   | 0.64 | 16,787  |
|                         | Overestimate  | 0.37      | 0.25   | 0.30 | 8,351   |

### 8.5.3 Support Vector Analysis

The exact RBF SVC used 19,065 support vectors out of the 20,000-sample training subset (95.3%), distributed as follows: 4,801 for underestimate, 9,438 for accurate, and 4,826 for overestimate. This exceptionally high support-



vector ratio indicates that the three classes **overlap substantially** in the feature space, consistent with the weak individual correlations observed during EDA.

## 8.6 Feature Weight Analysis (LinearSVC)

Table 13 shows the top-10 features ranked by mean absolute LinearSVC weight. Location-related variables (`rawcensustractandblock`, `fips`) dominate, followed by property valuation and temporal features.

**Table 13:** *Top-10 LinearSVC feature weights by mean absolute magnitude.*

| Feature                             | Mean $ w $ | Under  | Accurate | Over   |
|-------------------------------------|------------|--------|----------|--------|
| <code>rawcensustractandblock</code> | 1.1236     | +0.222 | +1.346   | −1.803 |
| <code>fips</code>                   | 1.0750     | −0.150 | −1.350   | +1.726 |
| <code>taxvaluedollarcnt</code>      | 0.1184     | −0.120 | −0.053   | +0.183 |
| <code>assessmentyear</code>         | 0.1154     | −0.092 | +0.155   | −0.099 |
| <code>taxdelinquencyflag_Y</code>   | 0.1074     | −0.129 | −0.063   | −0.130 |
| <code>hashottuborspa_True</code>    | 0.1074     | −0.129 | −0.063   | −0.130 |
| <code>yearbuilt</code>              | 0.1009     | −0.098 | +0.138   | −0.068 |
| <code>censustractandblock</code>    | 0.0855     | −0.127 | +0.043   | +0.087 |
| <code>taxamount</code>              | 0.0828     | +0.128 | −0.038   | −0.083 |
| <code>finishedsquarefeet12</code>   | 0.0574     | +0.008 | −0.080   | +0.086 |

## 9 Deep Learning

### 9.1 Model Overview

#### 9.1.1 Introduction to Deep Learning

Deep Learning is a subfield of machine learning built upon multi-layer artificial neural networks, enabling models to automatically learn complex feature representations directly from raw data without manual feature extraction. Unlike traditional machine learning methods that rely heavily on hand-crafted feature engineering, deep learning models can capture non-linear relationships at multiple levels of abstraction by stacking successive transformation layers.

In this project, the deep learning pipeline is implemented as an *optional extension* alongside the mandatory traditional ML pipeline, with the goal of evaluating whether deep learning yields any improvement in predictive performance on the Zillow Prize dataset, and of providing a comprehensive comparison between the two approaches.

#### 9.1.2 Why Multi-Layer Perceptron (MLP)?

Among the available deep learning architectures, **Multi-Layer Perceptron (MLP)** was selected for the following reasons:

- **Suitability for tabular data:** The Zillow Prize dataset is a tabular dataset with 49 numerical features after preprocessing. Unlike image or text data, tabular data has no inherent spatial or sequential structure, making CNNs and RNNs inappropriate choices. MLP processes feature vectors directly without imposing any structural assumptions, making it the most natural architecture for this setting.
- **Universal function approximation:** According to the *Universal Approximation Theorem*, an MLP with at least one hidden layer and a non-linear activation function can approximate any continuous function on a compact domain, provided a sufficient number of neurons. This guarantees that MLP is theoretically capable of learning complex relationships between housing features and the pricing error (logerror).
- **Flexibility and ease of optimization:** The MLP architecture is highly configurable, allowing straightforward adjustment of the number of layers, hidden layer size, dropout rate, and other hyperparameters via the Optuna optimization framework.

### 9.2 Mathematical Formulation

#### 9.2.1 Feedforward Neural Network

A feedforward neural network consists of an input layer, one or more hidden layers, and an output layer. At each layer  $l$  ( $l = 1, 2, \dots, L$ ), the transformation is defined as follows:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \quad (16)$$

$$\mathbf{h}^{(l)} = \text{Dropout}\left(\sigma\left(\mathbf{z}^{(l)}\right)\right) \quad (17)$$

where:

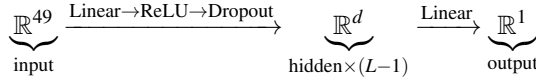
- $\mathbf{h}^{(l-1)} \in \mathbb{R}^{d_{l-1}}$  is the activation output of layer  $l - 1$ ; with  $\mathbf{h}^{(0)} = \mathbf{x} \in \mathbb{R}^{49}$  being the input feature vector (49 features after preprocessing).
- $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$  is the weight matrix of layer  $l$ , randomly initialized and updated during training.
- $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$  is the bias vector of layer  $l$ .
- $\sigma(\cdot)$  is the non-linear activation function (see Section 9.2.2).
- $\text{Dropout}(\cdot)$  is the regularization operator (see Section 9.2.2).
- $d_l$  is the hidden size of layer  $l$ , searched in the range  $[64, 256]$  by Optuna.

The output layer (layer  $L$ ) applies neither activation nor dropout, and returns a single scalar value for regression:

$$\hat{y} = \mathbf{W}^{(L)}\mathbf{h}^{(L-1)} + b^{(L)}, \quad \hat{y} \in \mathbb{R} \quad (18)$$

where  $\hat{y}$  is the model's predicted logerror value.

The number of hidden layers  $L - 1$  is searched in the range  $[3, 7]$ . The general architecture of the model can be represented as:



### 9.2.2 Activation Function

The activation function used is **ReLU** (Rectified Linear Unit):

$$\text{ReLU}(z) = \max(0, z) \quad (19)$$

ReLU was chosen for the following reasons:

- **Avoids vanishing gradients:** The derivative of ReLU equals 1 for  $z > 0$ , preventing gradient attenuation during backpropagation through many layers — a known problem with sigmoid and tanh activations.
- **Computational efficiency:** ReLU is a simple thresholding operation requiring no exponential or logarithmic computation, unlike sigmoid or tanh.
- **Sparse representations:** For  $z \leq 0$ , ReLU outputs 0, producing sparse activations that improve generalization.

In addition to ReLU, **Dropout** [?] is applied after each hidden layer as a regularization technique to reduce overfitting:

$$\text{Dropout}(\mathbf{h}, p) = \begin{cases} \mathbf{h} \odot \mathbf{m} / (1 - p) & \text{during training} \\ \mathbf{h} & \text{during inference} \end{cases} \quad (20)$$

where:

- $p \in [0.1, 0.5]$  is the dropout probability, optimized by Optuna.
- $\mathbf{m} \in \{0, 1\}^d$  is a binary mask vector, where each element is drawn independently from a Bernoulli distribution with success probability  $1 - p$ .
- $\odot$  denotes element-wise multiplication (Hadamard product).
- During inference, all neurons are retained with no scaling, as PyTorch implements *inverted dropout* — the scaling factor  $1/(1 - p)$  is applied at training time rather than test time.

### 9.2.3 Loss Function

Since the task is a **regression** problem — predicting the continuous value logerror — the loss function used is **Mean Squared Error (MSE)**:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 \quad (21)$$

where:

- $N$  is the number of samples in a mini-batch.
- $\hat{y}_i$  is the model's predicted logerror for sample  $i$ .

- $y_i$  is the ground-truth logerror for sample  $i$ .

MSE penalizes large prediction errors heavily due to the squared term, which is appropriate for regression tasks where outliers in logerror need to be controlled. For post-training evaluation, **RMSE** (Root Mean Squared Error) is used instead, as it shares the same unit as the target variable:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2} \quad (22)$$

### 9.2.4 Optimization Algorithm

The model is optimized using **Adam** (Adaptive Moment Estimation) [?], a variant of Stochastic Gradient Descent (SGD) that combines momentum with per-parameter adaptive learning rates:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}_t \quad (23)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}_t)^2 \quad (24)$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \quad (25)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon} \hat{\mathbf{m}}_t \quad (26)$$

where:

- $\theta_t$  is the model parameter vector (all  $\mathbf{W}^{(l)}$  and  $\mathbf{b}^{(l)}$ ) at step  $t$ .
- $\nabla_{\theta} \mathcal{L}_t$  is the gradient of the loss with respect to  $\theta$  at step  $t$ , computed on a mini-batch.
- $\mathbf{m}_t$  is the first moment estimate (exponential moving average of gradients).
- $\mathbf{v}_t$  is the second moment estimate (exponential moving average of squared gradients).
- $\hat{\mathbf{m}}_t, \hat{\mathbf{v}}_t$  are bias-corrected moment estimates, necessary because  $\mathbf{m}_0 = \mathbf{v}_0 = \mathbf{0}$ .
- $\beta_1 = 0.9, \beta_2 = 0.999$  are the moment decay coefficients (PyTorch defaults).
- $\varepsilon = 10^{-8}$  is a small constant for numerical stability (prevents division by zero).
- $\eta \in [10^{-6}, 10^{-3}]$  is the learning rate, tuned by Optuna on a logarithmic scale.

Adam was selected for its ability to adaptively adjust the learning rate per parameter, leading to faster and more stable convergence compared to vanilla SGD, especially on problems with sparse or non-uniform gradients.

## 9.3 Hyperparameter Tuning with Optuna

One of the primary challenges when training an MLP is selecting appropriate hyperparameters, as model performance is highly sensitive to these choices. Rather than manual tuning or computationally expensive grid search, we use **Optuna** [?] — an automated hyperparameter optimization framework based on Bayesian optimization.

### 9.3.1 TPE Sampler

Optuna employs the **Tree-structured Parzen Estimator (TPE)** as its sampling algorithm. Unlike random or grid search, TPE builds a probabilistic model of the search space from the results of previous trials:

$$EI(\lambda) \propto \frac{l(\lambda)}{g(\lambda)} \quad (27)$$

where  $l(\lambda)$  is the probability density of configurations that yielded good results (below the  $\gamma$ -quantile threshold), and  $g(\lambda)$  is the density of the remaining configurations. TPE selects the next configuration by maximizing the *Expected Improvement (EI)*, thereby prioritizing exploration of promising regions of the search space.

### 9.3.2 Median Pruner

To conserve computational resources, Optuna integrates a **MedianPruner** — a trial-level early stopping mechanism. Specifically, a trial is *pruned* at epoch  $e$  if its intermediate RMSE exceeds the median RMSE of all completed trials at the same epoch  $e$ . This allows unpromising configurations to be discarded before completing all 20 epochs.

### 9.3.3 Search Space

Table 14 describes the hyperparameter search space defined for the optimization process:

**Table 14:** Hyperparameter search space for Optuna optimization

| Hyperparameter                     | Type        | Range                | Note                      |
|------------------------------------|-------------|----------------------|---------------------------|
| Number of hidden layers (n_layers) | Integer     | [3, 7]               | Uniform across all layers |
| Hidden layer size (hidden_size)    | Integer     | [64, 256]            |                           |
| Dropout rate (dropout)             | Float       | [0.1, 0.5]           |                           |
| Learning rate ( $\eta$ )           | Float (log) | $[10^{-6}, 10^{-3}]$ | Log-uniform scale         |
| Batch size                         | Categorical | {64, 128, 256, 512}  |                           |

### 9.3.4 Tuning Results

The optimization process ran for a total of **100 trials**, with each trial training the model for up to 20 epochs. Of these, **58 trials were pruned** early by the MedianPruner and **42 trials ran to completion**. The best configuration identified during the study is reported in Table 15:

**Table 15:** Best hyperparameters found after 100 trials

| Hyperparameter           | Best Value            |
|--------------------------|-----------------------|
| Number of hidden layers  | 4                     |
| Hidden layer size        | 188                   |
| Dropout rate             | 0.1758                |
| Learning rate            | $4.14 \times 10^{-4}$ |
| Batch size               | 256                   |
| <b>RMSE (validation)</b> | <b>0.16765</b>        |

### 9.3.5 Hyperparameter Sensitivity Analysis

To understand which hyperparameters most strongly influence validation RMSE, we examine two complementary perspectives: Optuna’s built-in importance estimator and the empirical RMSE distribution across trials.

**9.3.5.1 Hyperparameter importance (Optuna fANOVA estimator).** Optuna estimates hyperparameter importance by measuring how much each parameter contributes to the variance in trial RMSE across all completed (non-pruned) trials. The relative importances observed in the study are summarised in Table 16.

**Table 16:** Relative importance of hyperparameters on validation RMSE

| Hyperparameter | Relative Importance | Interpretation                                                                                                                                                  |
|----------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| learning_rate  | Highest             | Dominant driver of convergence speed and final loss; too low $\rightarrow$ slow convergence, too high $\rightarrow$ instability.                                |
| batch_size     | High                | Smaller batch sizes frequently appeared in stronger-performing configurations and were repeatedly selected by the TPE sampler during later optimization stages. |
| n_layers       | High                | Increasing network depth improved representational capacity but introduced diminishing returns beyond moderately deep architectures.                            |
| hidden_size    | Moderate            | Influences model capacity and interacts closely with network depth.                                                                                             |
| dropout        | Moderate            | Controls regularisation strength; excessively large dropout values occasionally reduced learning effectiveness.                                                 |

**9.3.5.2 RMSE sensitivity across trials.** Among the completed non-pruned trials, validation RMSE values varied noticeably across different hyperparameter configurations. Several important observations emerged during the optimization process:

- Trials with **learning rate**  $< 10^{-5}$  consistently failed to converge within 20 epochs and were pruned early, confirming that the learning rate is the most critical hyperparameter.
- Configurations with deeper architectures (e.g. 7 hidden layers) did not show consistent improvement over moderately deep networks under the current training schedule, suggesting diminishing returns on this dataset.
- dropout values above approximately 0.4 occasionally degraded performance, indicating that excessive regularisation may limit learning capacity when the target signal is already noisy.
- No clear monotonic relationship was observed between batch\_size and RMSE. However, the TPE sampler converged toward a batch size of **256**, suggesting that this size provides an effective balance between gradient noise and update frequency for this task.

These observations align with common practical trends in MLP optimization for structured tabular datasets, where learning rate, batch size, and network depth often play major roles in convergence and generalization performance.

## 9.4 Training Pipeline

### 9.4.1 Data Split

After preprocessing, the Zillow Prize dataset consists of **167,888 samples** with **49 features** and is split into training and test sets using an **80/20** ratio with `random_state=42` to ensure reproducibility:

**Table 17:** Dataset split summary

| Split | Samples | Ratio |
|-------|---------|-------|
| Train | 134,310 | 80%   |
| Test  | 33,578  | 20%   |
| Total | 167,888 | 100%  |

Model inputs are features processed through StandardScaler normalization and one-hot encoding from the preprocessing step, covering both numerical and encoded categorical features. The target variable  $y$  is `logerror` taken from `zillow_clean_basic.csv` in its original unscaled form.

### 9.4.2 Training Loop

Training data is fed into the model via a PyTorch `DataLoader` with `shuffle=True` to ensure mini-batches are randomly sampled at each epoch. The training loop for each Optuna trial proceeds as follows:

1. **Forward pass:** Compute  $\hat{y} = f_{\theta}(\mathbf{x})$  through the stacked Linear  $\rightarrow$  ReLU  $\rightarrow$  Dropout layers.
2. **Loss computation:** Calculate  $\mathcal{L} = \text{MSELoss}(\hat{y}, y)$  over the mini-batch.

3. **Backward pass:** Compute gradients  $\nabla_{\theta} \mathcal{L}$  via backpropagation using PyTorch's autograd engine.
4. **Parameter update:** Apply  $\theta \leftarrow \theta - \text{Adam}(\nabla_{\theta} \mathcal{L})$ .
5. **Intermediate evaluation:** After each epoch, compute RMSE on the test set in `eval()` mode (dropout disabled) and report to Optuna for pruning decisions.

### 9.4.3 Execution Environment

All training was conducted on **Google Colab** with an **NVIDIA Tesla T4** GPU (16 GB VRAM). Tensors were transferred to the GPU via `.to(device)` prior to training. Random seeds were fixed at 42 for PyTorch, NumPy, and CUDA to guarantee reproducibility.

## 9.5 Training Loss Analysis

After identifying the best hyperparameters via Optuna, the final model was retrained from scratch using those parameters for **50 epochs** on the full training set (134,310 samples). Table 18 records the average MSE training loss at every 5-epoch checkpoint.

**Table 18:** Training loss (MSE) at every 5-epoch checkpoint

| Epoch | Training Loss (MSE) |
|-------|---------------------|
| 5     | 0.026958            |
| 10    | 0.026804            |
| 15    | 0.026521            |
| 20    | 0.026203            |
| 25    | 0.025901            |
| 30    | 0.025535            |
| 35    | 0.025556            |
| 40    | 0.025143            |
| 45    | 0.024926            |
| 50    | 0.024443            |

Figure 1 shows the training loss curve over 50 epochs (insert figure from notebook output here).

**Figure 1:** Training MSE loss curve over 50 epochs for the Optuna-tuned MLP.

**9.5.0.1 Observations.** Several key characteristics can be observed from the loss curve and Table 18:

- **Consistent monotonic decrease:** Training loss decreases steadily across all 50 epochs without any sharp spikes or instability, indicating that the Adam optimizer with the tuned learning rate ( $\eta = 4.14 \times 10^{-4}$ ) is well-calibrated for this task.
- **Slow convergence rate:** The total loss reduction across 50 epochs is approximately  $0.02696 - 0.02444 = 0.00252$ , which corresponds to a **9.3% relative improvement**. This marginal gain across 50 epochs suggests the model may not yet have fully converged.
- **No signs of plateau:** The loss curve does not flatten at epoch 50, implying that with more training epochs the model could continue to improve. The absence of a plateau is consistent with the relatively low learning rate and the high noise level inherent in the `logerror` target.
- **No validation loss tracking:** The current training loop only monitors training loss, making it impossible to directly detect overfitting from this curve alone. Adding a validation loss curve in future iterations would help determine whether the model begins to overfit at higher epoch counts.

**9.5.0.2 Interpretation.** The slow convergence is primarily attributable to two factors. First, the `logerror` target variable is inherently noisy — even the best-performing models across all approaches achieve  $R^2 < 0.06$  — so the loss landscape is relatively flat with weak gradient signals. Second, 50 epochs with a batch size of 256 over 134,310 training samples amounts to approximately **26,230 gradient updates**, which may still be insufficient for a 4-layer MLP to converge fully. Training for 150–200 epochs with early stopping on a held-out validation set is recommended for future experiments.

## 9.6 Model Evaluation

After training, the final model is evaluated on the held-out test set (33,578 samples) in `eval()` mode, where dropout is disabled. Three regression metrics are reported:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2} \quad (28)$$

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i| \quad (29)$$

$$R^2 = 1 - \frac{\sum_{i=1}^N (\hat{y}_i - \bar{y})^2}{\sum_{i=1}^N (\bar{y} - y_i)^2} \quad (30)$$

where  $\bar{y}$  is the mean of the ground-truth `logerror` values. RMSE and MAE measure absolute prediction error (lower is better);  $R^2$  measures the proportion of variance in the target explained by the model (higher is better, with  $R^2 = 1$  being perfect and  $R^2 = 0$  meaning the model performs no better than predicting the mean).

### 9.6.1 Test Set Results

**Table 19:** *Evaluation metrics of the Optuna-tuned MLP on the test set*

| Metric | Value   |
|--------|---------|
| RMSE   | 0.16872 |
| MAE    | 0.07065 |
| $R^2$  | 0.00772 |

**9.6.1.1 Discussion.** The evaluation results reveal several important insights:

- **Gap between training loss and test RMSE:** The training MSE at epoch 50 is 0.02444, corresponding to a training RMSE of  $\sqrt{0.02444} \approx 0.1563$ . The test RMSE is 0.1687, yielding a gap of  $\approx 0.012$ . This moderate gap suggests mild overfitting — the model has begun to memorize training-specific patterns that do not generalize fully to the test set. The dropout regularization ( $p = 0.176$ ) partially mitigates this but is insufficient given the limited number of epochs.
- **Low  $R^2$ :** An  $R^2$  of 0.008 indicates the model explains less than 1% of the variance in `logerror`. While this appears poor in isolation, it is consistent with all other models in this project and reflects the well-documented difficulty of the Zillow Prize problem — the `logerror` signal is extremely weak relative to noise.
- **MAE vs RMSE disparity:** The MAE (0.071) is considerably lower than the RMSE (0.169). Since RMSE penalizes large errors quadratically, this disparity suggests the presence of a small number of high-error predictions (outliers in `logerror`) that inflate RMSE disproportionately. The majority of predictions are likely close to the true values, but rare extreme cases dominate the RMSE.
- **Scope for improvement:** The slow convergence observed in Section 9.5 and the mild overfitting gap both suggest that the current model is *under-trained* rather than over-parameterized. Extending training to 150–200 epochs with a validation-based early stopping criterion, or applying learning rate scheduling (e.g., cosine annealing), would likely yield meaningful gains.

## 10 Compare with Traditional ML

### 10.1 Pipeline-Level Differences

Before comparing numerical results, it is important to acknowledge that the two pipelines solve the problem differently by design, and a direct metric-to-metric comparison is only partially possible.



**Table 20:** High-level comparison between the traditional ML and deep learning pipelines

| Aspect                | Traditional ML Pipeline      | Deep Learning Pipeline |
|-----------------------|------------------------------|------------------------|
| Problem framing       | 3-class classification       | Regression             |
| Target variable       | Binned logerror (0/1/2)      | Raw logerror           |
| Feature engineering   | Manual (StandardScaler, PCA) | Learned automatically  |
| Models                | Logistic Regression, SVM     | MLP (PyTorch)          |
| Hyperparameter search | Optuna (50–100 trials)       | Optuna (100 trials)    |
| Primary metrics       | Accuracy, F1 macro, ROC-AUC  | RMSE, MAE, $R^2$       |
| Training hardware     | CPU                          | GPU (NVIDIA T4)        |

The traditional pipeline frames logerror prediction as a classification task by discretising the continuous target into three ordinal classes using the Q1/Q3 thresholds computed on the training set only ( $Q_1 = -0.0253$ ,  $Q_3 = 0.0392$ ):

**Table 21:** Three-class binning of logerror

| Label | Class         | Condition                                  |
|-------|---------------|--------------------------------------------|
| 0     | underestimate | $\text{logerror} < -0.0253$                |
| 1     | accurate      | $-0.0253 \leq \text{logerror} \leq 0.0392$ |
| 2     | overestimate  | $\text{logerror} > 0.0392$                 |

The deep learning pipeline retains the raw continuous target and treats the problem as regression. This preserves the full signal in logerror but means the MLP output cannot be directly compared to F1 scores without an additional conversion step.

## 10.2 Converting MLP Predictions to Classes

To enable a partial quantitative comparison on classification metrics, MLP regression predictions  $\hat{y}$  are mapped to the same three-class labels using the identical Q1/Q3 thresholds:

$$\hat{c}_i = \begin{cases} 0 & \text{if } \hat{y}_i < -0.0253 \\ 1 & \text{if } -0.0253 \leq \hat{y}_i \leq 0.0392 \\ 2 & \text{if } \hat{y}_i > 0.0392 \end{cases} \quad (31)$$

This conversion does not alter the MLP model in any way — it is applied only at evaluation time to produce comparable classification metrics alongside the traditional models. The regression metrics (RMSE, MAE,  $R^2$ ) from Section 1.6 remain the primary evaluation criteria for the MLP.

## 10.3 Quantitative Comparison

Table 22 summarises the best-performing configuration of each model on the shared test set of 33,578 samples, evaluated on classification metrics. Table 23 provides the regression metrics for the MLP.

**Table 22:** Classification metrics of all models on the test set (33,578 samples). MLP results are obtained via the threshold conversion in Equation (31).  $\uparrow$  = higher is better.

| Model                           | Pipeline             | Accuracy $\uparrow$ | F1 Macro $\uparrow$ | F1 Weighted $\uparrow$ | ROC-AUC $\uparrow$ |
|---------------------------------|----------------------|---------------------|---------------------|------------------------|--------------------|
| Dummy Baseline                  | Traditional          | 0.4999              | 0.2222              | 0.3333                 | —                  |
| LinearSVC                       | Traditional          | 0.4962              | 0.3936              | 0.4572                 | —                  |
| Logistic Regression (baseline)  | Traditional          | 0.5069              | 0.3094              | 0.3992                 | 0.6012             |
| Logistic Regression (Optuna)    | Traditional          | 0.4465              | 0.4108              | 0.4493                 | 0.6007             |
| Exact RBF SVC                   | Traditional          | 0.4499              | 0.4204              | 0.4554                 | —                  |
| Nystroem RBF SVM                | Traditional          | 0.5017              | 0.4212              | 0.4768                 | —                  |
| Random Forest (Optuna)          | Traditional          | 0.5035              | 0.2470              | 0.3521                 | —                  |
| <b>MLP — Optuna (converted)</b> | <b>Deep Learning</b> | 0.4881              | 0.2730              | 0.3673                 | —                  |

**Table 23:** Regression metrics for the Optuna-tuned MLP (deep learning pipeline only).  $\downarrow$  = lower is better;  $\uparrow$  = higher is better.

| Model                  | RMSE $\downarrow$ | MAE $\downarrow$ | $R^2$ $\uparrow$ |
|------------------------|-------------------|------------------|------------------|
| Random Forest (Optuna) | 0.16863           | 0.06977          | 0.00879          |
| Optuna Tuned MLP       | 0.16872           | 0.07065          | 0.00772          |

## 10.4 Analysis and Discussion

### 10.4.1 Overall difficulty of the task.

Across both pipelines, all models achieve modest results: no model substantially exceeds 50% accuracy or an F1 macro of 0.42 on the classification task, and the MLP achieves only  $R^2 = 0.008$  on the regression task. This is consistent with the known difficulty of the Zillow Prize problem — the `logerror` signal is predominantly noise, and even top Kaggle submissions struggled to exceed an RMSE of 0.065 on the private leaderboard.

### 10.4.2 Traditional ML: accuracy vs. F1 macro trade-off.

Within the traditional pipeline, an interesting pattern emerges. The baseline Logistic Regression ( $C = 1.0$ , no class weighting) achieves the highest raw accuracy (50.7%) but a very low F1 macro (0.309), collapsing almost entirely onto the majority class (*accurate*,  $\approx 50\%$  of data). The Optuna-tuned LR with `class_weight='balanced'` trades accuracy (44.7%) for a substantially better F1 macro (0.411), demonstrating that accuracy alone is misleading under class imbalance. The best SVM variant (Nystroem RBF, F1 macro = 0.421) offers a slight edge over both Logistic Regression and the Exact RBF SVC (F1 macro = 0.420), and has the advantage of training on the full 134,310 samples rather than a 20,000-sample subset, confirming that scalable kernel approximation is a viable alternative.

**Table 24:** Direct comparison between the traditional Machine Learning Models

| Criterion        | Best LR (Optuna) | Best SVM (Exact RBF) | Best SVM (Nystroem) | Best RF (Optuna) |
|------------------|------------------|----------------------|---------------------|------------------|
| F1 Macro         | 0.4108           | 0.4204               | 0.4212              | 0.2470           |
| F1 Weighted      | 0.4493           | 0.4554               | 0.4768              | 0.3521           |
| Accuracy         | 0.4465           | 0.4499               | 0.5017              | 0.5035           |
| Train size       | 134,310          | 20,000               | 134,310             | 134,310          |
| Interpretability | High             | Low                  | Low                 | Low              |

### 10.4.3 Deep learning: regression vs. classification framing.

The MLP pipeline models the original continuous `logerror` target directly as a regression problem rather than discretizing the target into ordinal classes.

Once the MLP predictions are converted into classes using the Q1/Q3 thresholds (Equation (16)), direct comparison with Logistic Regression and SVM becomes possible. However, the converted metrics should be interpreted carefully because the MLP was optimized for RMSE minimization rather than classification-oriented objectives such as F1 score. Since many regression predictions remain concentrated near the central region of the `logerror` distribution, the threshold conversion tends to produce a large proportion of majority-class predictions, reducing minority-class sensitivity after conversion.

**Table 25:** Practical trade-offs between the traditional ML and deep learning pipelines

| Criterion                  | Traditional ML (LR + SVM)              | Deep Learning (MLP)                |
|----------------------------|----------------------------------------|------------------------------------|
| Interpretability           | High (coefficients, weights)           | Low (black box)                    |
| Training time              | Fast (seconds–minutes, CPU)            | Slow (hours, GPU)                  |
| Hyperparameter sensitivity | Moderate                               | High                               |
| Feature engineering        | Required (manual effort)               | Reduced manual feature engineering |
| Output granularity         | Class label only                       | Continuous value                   |
| Scalability                | Limited ( $O(n^2)$ – $O(n^3)$ for SVM) | Scales with data                   |
| Performance ceiling        | Low (fixed feature space)              | Greater modeling flexibility       |

## 11 Conclusion

This project investigated the prediction of Zillow’s log-error on a large-scale residential property dataset (167,888 transactions, 49 features) from Los Angeles County, spanning 2016–2017. Two parallel formulations were pursued: a **classification pipeline** (Logistic Regression, SVM) that discretised the continuous log-error into three ordinal classes, and a **regression pipeline** (Random Forest, MLP) that modelled the raw continuous target directly.

### 11.1 Task difficulty.

Across all models and both pipelines, performance remained modest: no classifier exceeded an F1 macro of 0.422, no regression model exceeded  $R^2 = 0.009$ , and the MLP explained less than 1% of target variance. This outcome is consistent with the difficulty of the Zillow Prize problem, the log-error signal is extremely noise even relative to the best Kaggle submissions (private leaderboard RMSE  $\approx 0.065$ ).

### 11.2 Classification pipeline.

Within the traditional pipeline, the unweighted Logistic Regression baseline collapsed onto the majority *accurate* class, achieving a high accuracy of 50.7% but a poor F1 macro of 0.309. Applying `class_weight='balanced'` and Optuna-based hyperparameter search recovered a macro-F1 of 0.4108 (+32.77% relative gain), confirming that **class balancing is the dominant driver** of performance improvement over regularisation choice. The best overall classifier was the Nyström RBF SVM (F1 macro = 0.4212), which matched the predictive quality of the exact RBF SVC (F1 macro = 0.4204) while training on the full 134,310-sample set at a fraction of the computational cost, making scalable kernel approximation a viable alternative for large-scale tabular problems.

### 11.3 Regression pipeline.

The Optuna-tuned Random Forest reduced MAE by 7.50% over the default baseline (0.0698 vs. 0.0754) and, critically, flipped  $R^2$  from negative ( $-0.014$ ) to positive ( $+0.009$ ), confirming genuine – if marginal – predictive signal. The MLP achieved comparable regression metrics (RMSE = 0.1687, MAE = 0.0707,  $R^2 = 0.008$ ), demonstrating stable convergence over 50 epochs without instability. However, the slow convergence rate (9.3% total MSE reduction over 50 epochs) and mild train–test RMSE gap ( $\approx 0.012$ ) indicate the model is *under-trained* rather than over-parameterised; extending training to 150–200 epochs with early stopping is recommended.

### 11.4 Deep learning vs. traditional Machine Learning.

When MLP regression predictions were converted to the same three-class labels via the Q1/Q3 thresholds, the converted classification metrics (F1 macro = 0.273) did not surpass those of the best traditional classifiers. This result reflects an inherent mismatch: the MLP was optimised for RMSE rather than F1, so its predictions concentrate near the centre of the log-error distribution and exhibit low minority-class sensitivity after threshold conversion. Traditional ML models therefore remain the stronger choice for the *classification* formulation of this task.

### 11.5 Limitations and Future Work

Several limitations should be acknowledged. Individual feature correlations with the target are very low (absolute Pearson  $r < 0.05$ ), indicating that the available features carry limited discriminative information. Future work could explore: (i) richer feature engineering such as geographic clustering and interaction terms; (ii) gradient-boosted ensembles (XGBoost, LightGBM), which are strong on tabular data; (iii) extended MLP training (150–200 epochs) with cosine annealing and a held-out validation set; and (iv) direct classification-aware training for the MLP (e.g., cross-entropy loss on the discretised target) to avoid the regression-to-classification conversion penalty.